

Face Search API

Lets an attendee find themselves in an event's gallery by submitting a selfie. This is an addition to the mobile gallery API doc previously shared — read that first for how to authenticate and get a bearer token, and for the general access rules (organization membership, 403 / 404 behavior). Everything below assumes that same token.

1. Before you call this

Face search is a feature an organization can turn on or off. If it's off for the organization that owns the event, the endpoint returns 403 — treat that as "not available for this event," not as a bug. There's no separate endpoint to check the flag in advance, so the simplest approach is: call it, and hide/disable the "find my photos" entry point in your UI if you get that specific 403 back.

2. How it works, briefly

Organizers' uploaded photos are automatically scanned for faces in the background — no action needed from your app. When a user submits a selfie, the server compares it against every face found in that event's photos and returns the photos that look like a match, ranked best-first.

The selfie itself isn't saved anywhere — it's used to do the one comparison and then discarded.

3. POST /events/:id/face-search

Send the selfie as a file in a multipart/form-data body, under the field name image .

```
POST /events/{eventId}/face-search?strictness=normal
Authorization: Bearer <token>
Content-Type: multipart/form-data

image: <selfie file>
```

Example (curl):

```
curl -X POST "https://api.pix.dx5ve.com/events/EVENT_ID/face-search?strictness=normal" \
-H "Authorization: Bearer $TOKEN" \
-F "image=@selfie.jpg"
```

Request

Field	Where	Required	Notes
<code>image</code>	form field	yes	One photo of the user's face. Max size 15 MB. Common formats (JPEG, PNG, HEIC) all work.
<code>strictness</code>	query param	no	<code>strict</code> , <code>normal</code> (default), or <code>loose</code> — see below.

Strictness controls how confident a match needs to be to show up:

- `strict` — fewest results, but the ones you get are more likely to be correct.
- `normal` — a balanced default, good for most cases.
- `loose` — more results, but with a higher chance of including photos that aren't actually the person.

Response — success (200)

```
{
  "matches": [
    {
      "photoId": "uuid",
      "matchConfidencePercent": 87,
      "thumbnailUrl": "https://... or null",
      "caption": "string or null",
      "sessionId": "uuid or null",
      "sessionName": "string or null",
      "day": "2026-07-15"
    }
  ]
}
```

- `matches` is sorted best match first, and capped at 100 results.
- `matchConfidencePercent` (0–100) is a relative score for ranking — use it to sort/highlight results, not as a precise statistical guarantee. Avoid UI copy like "87% chance this is you."
- `thumbnailUrl` is a temporary link that expires after about an hour, same as photo URLs elsewhere in the gallery API — don't cache it long-term. If it's expired, re-run the search or re-fetch the photo from the regular gallery endpoint.
- An empty `matches` list is a normal, successful result — it just means nothing matched closely enough. Show that as "no matches found," not as an error.

Response — errors

Status	Body example	What it means
400	<pre>{ "error": "Missing image" }</pre>	No image was included in the request.
403	<pre>{ "error": "Face search is not enabled for this organization" }</pre>	This event's organization doesn't have face search turned on. See §1.
403 / 404	—	You don't have access to this event, same rules as any other event endpoint.
422	<pre>{ "error": "Could not read the submitted image" }</pre>	The file wasn't a readable image (corrupted or unsupported).
422	<pre>{ "error": "No clear face detected in the submitted photo" }</pre>	The image was fine, but no face could be found in it clearly enough to search with. Ask the user to retake it with better lighting and their face clearly visible.
429	rate-limit error	Too many requests too quickly — see below.
502	<pre>{ "error": "Face detection service unavailable" }</pre>	A temporary backend issue. Safe to retry after a short delay.

If the selfie happens to contain more than one face, the app doesn't ask which one to use — it automatically searches with whichever face it's most confident about. For best results, have the user submit a selfie of just themselves.

Rate limiting

This endpoint allows **10 requests per minute per logged-in user**. It's intentionally stricter than the rest of the API because it deals with a photo of someone's face. Disable your submit button while a request is in flight so users don't accidentally trigger repeat requests and hit this limit.

4. Tips for building on this

- A tightly-cropped, well-lit selfie with the face clearly visible works best — avoid group photos or distant shots.
- Show a loading indicator; this call takes a bit longer than a normal gallery fetch since it has to analyze the image first.

- Treat "no matches" and "error" as different states in your UI — the former is a valid, expected outcome (especially with `strict`).
- Don't store a copy of the selfie in your own app/backend beyond what's needed to send the request — the server doesn't keep one, and your app shouldn't either.

Known limitations

- Match strictness levels aren't finely tuned yet and may be adjusted over time.
- There's no way to tell the API which face to use if a selfie has multiple people in it.
- There's no endpoint to check ahead of time whether face search is enabled for a given event — you'll only find out from a `403` when you call it.